

筆記:樹搜尋 harness v1 → v4 的演進

一句話總綱:每一版都不動前一版(import 繼承),只加新東西 → 每版都是前一版的**嚴格超集**。這樣舊實驗永遠可重現,新功能也能單獨歸因。四版對應四個不同的問題。

Contents

一眼版	1
v1 — 最小骨架	1
v2 — 集成一等公民 + 4 建議	2
v3 — 預算 / 收尾 / 生產化 6 件	2
v4 — 外部想法注入(ERA 第二支柱 / 階段 5)	2
直向看:哪個機制屬於哪一版	3
一個重要後話:v4 之後沒有 v5	3
三個要記住的重點	3

一眼版

版	檔	行數	核心 = 新增了什麼	回答的問題	用在
v1	<code>harness.py</code>	252	最小候選樹骨架	「線性單路徑 → 搜尋樹?」	s3e9 / s3e14 / s3e5
v2	<code>harness_v2.py</code>	376	集成一等公民 + 4 建議	「blend 怎麼放進樹?離散指標怎麼辦?」	s3e3 / s3e7 / s3e1 / s3e19 / s3e11
v3	<code>harness_v3.py</code>	856	預算 / 收尾 / 生產化 6 件	「跑久了會空轉、卡死、resume 壞掉」	跨季 5 場 + 部分 S3 回填
v4	<code>harness_v4.py</code>	540	外部想法注入([EXT])	「內部經驗用盡,怎麼引外部想法?」	階段 5 注入 / 重組

v1 — 最小骨架

是什麼:一棵「候選解的搜尋樹」。節點 = 一個完整解,血脈(lineage)= 從根長出的一條分支。

關鍵組件 - 節點: `{id, parent_id, mutation, config, score, status, wall_s}` - 選擇 `select_next_parent` + `PLATEAU_STREAK=3` 回溯(一條血脈連 3 個子節點沒改善 → 換血脈) - 每次 add 就存檔 → 崩潰安全 resume - 每場只需給一個 `evaluate(config) → score`

限制: `kind` 埋在 `config` 裡,沒有一等的 **blend** 支援(這正是 v2 要補的)。

v2 — 集成一等公民 + 4 建議

(來源:prototype 報告的「已知缺口 → 建議」)

1. **Ensemble-default node space**: `kind ("solo"/"blend")` 升為一等欄位;形式化 `cache_oof / load_oof + comp-agnostic eval_blend` (Dirichlet / 網格權重搜尋)。→ **鐵則**:後處理(取整 / `OptimizedRounder / clip`)必須寫進 `metric_fn` 內,才會對每組候選權重都套用。
2. **Metric-aware plateau**: `tie_rate` 偵測離散化分數面 → plateau 門檻 **3 → 5**、平手視為中性(不重置也不累加 streak)。
3. **config 去重**: `config_hash` (排序後 sha256)在 `add_node` 入口擋掉重複配方。
4. **經驗庫先驗** `suggest_priors` (關鍵字比對 `experience.md`,不呼叫 LLM)。

選擇 / 回溯規則原樣繼承 v1。

v3 — 預算 / 收尾 / 生產化 6 件

(把 driver 各自的臨時解法收成引擎預設)

1. **預算相位機** `init_budget / update_phase / should_stop`:總 **60 節點**, `exploit → explore_burst → stopped`;進 burst 後 15–20 評估沒刷新最佳就停。
2. **dedup 消耗預算**:同一父節點連撞 **2 次** → 燒一個 `status="failed"` 佔位(算進 `MAX_CHILDREN`、不算進 `n_evaluated`)→ 不空轉。
3. **solo 突破自動重開 blend 血脈**(新 solo 成全域最佳時,解凍已 plateau 的 blend 血脈去吸收它)。
4. **resume-state 持久化** `save/load_search_state + validate_state` 自檢(driver 狀態放 `tree["search_state"]`,不放模組全域)。
5. **子行程層級評估逾時** `eval_solo_subprocess ( signal.alarm` 殺不掉原生 `fit()`,要 OS 層級 kill;根因:一次 CatBoost 卡 28 分鐘)。
6. **burst 種子健全性閘**(長射程種子分數爛過門檻就標 plateau,不讓它衍生子節點)。

節點 schema / kind / 快取 / 去重 / plateau / 先驗全繼承 v2。

v4 — 外部想法注入(ERA 第二支柱 / 階段 5)

在 v3 搜尋核心上只加 [EXT] 注入路徑:

1. `parse_idea_bank()` 解析 `idea_bank.md` 的 [EXT-NN] 條目。
2. `parse_dedup_suppress_ids()` — 已被 [INT] 實證的 [EXT] 不重複觸發。
3. `suggest_ext_priors()` — [EXT] 版的 `suggest_priors`。
4. `suggest_priors_v4(mode=)` — `off` (只 [INT],逐位等於 v3)/ `ext ( [INT]+[EXT] )` 併池入口。

5. `recombine()` / `propose_recombinations()` — LLM 重組算子。

整個 v3 搜尋核心原樣繼承。

直向看:哪個機制屬於哪一版

機制	v1	v2	v3	v4
節點 / 血脈 / 回溯	●			
plateau streak=3	●	→adaptive 3/5		
solo	●			
blend / eval_blend / metric_fn		●		
config 去重		● 純擋	+燒預算	
經驗庫先驗 suggest_priors		●		
預算相位機 / mega-blend / stop			●	
resume / 子行程逾時 / sanity gate			●	
外部注入 [EXT] / LLM 重組				●

(●=該版新增;箭頭=在該版被改。空白=繼承前版不變。)

一個重要後話:v4 之後沒有 v5

v4 建了「想法進得來的門」([EXT] 注入 hook 正確),但當時消費端沒接—— `suggest_priors_v4` 回傳存進 `tree['priors']`、搜尋從沒讀。這就是 **write-only bug**(先驗從未真正注入搜尋)。

修復**不是 v5**,而是獨立的 `prior_wiring.py` (2026-07-13)把消費端接上,做成兩版對照: - **機械版**(規則模板翻譯, 決定性可重現) - **LLM in the loop 版**(LLM 讀先驗+戰況即時提案)

結論:同池重加權受凸最優上限約束(機械版 5/5 無增益),LLM 擴池注入去相關新成員在 2/5 場統計顯著。詳見 `docs/prior_wiring_findings.md`、`docs/bug_prior_injection_noop.md`。

記法:v4 = 門; `prior_wiring` = 門後的路。

三個要記住的重點

1. **演進哲學**:v1 骨架 → v2 「集成 + 離散指標正確性」 → v3 「跑得久、穩、生產化」 → v4 「想法來源擴到外部」。每版只加「上一批實戰**證據要求**的東西」,前版不動 → 舊結果永遠可重現。

2. **去重 v2 vs v3**:偵測方法一樣(`config_hash` 、同一入口點),差別是**撞到之後 v2 純擋、v3 連撞 2 次燒一格預算**(避免空轉)。
3. **v4 的 bug 教訓**:「有寫入、有日誌」≠「有被消費」——注入 hook 對,但消費端沒接就是空操作;驗收要驗到消費端(最後靠「開/關注入輸出逐位相同」抓到)。